

Real-Time Multi-Object Tracking with Custom Data Association and Motion Prediction

Mayank Goud Polagoni
College of Engineering
Northeastern University
Boston, MA, USA
polagoni.m@northeastern.edu

Kushwanth Vellanki
College of Engineering
Northeastern University
Boston, MA, USA
vellanki.ku@northeastern.edu

Abstract—We present a real-time multi-object tracking system that integrates YOLOv8n-based person detection with a custom-built tracking pipeline. Unlike approaches that rely on off-the-shelf trackers such as SORT or DeepSORT, our system implements tracking logic from scratch, including a Kalman Filter for motion prediction and the Hungarian algorithm for detection-to-track data association. The system supports two matching strategies: Intersection over Union (IoU) and Euclidean distance-based cost matrices. We evaluate performance across five experiments covering single versus multi-object scenarios, slow versus fast motion, IoU versus distance matching, partial and full occlusion handling, and real-time frame rate analysis. Results demonstrate stable identity assignment with consistent IDs across frames, accurate short-term motion prediction via the Kalman Filter, and real-time throughput of 5.9 to 8.9 FPS on a consumer laptop. Both matching strategies yield reliable tracking, with IoU mode running marginally faster. The system gracefully handles partial occlusion while acknowledging the expected limitation of ID reassignment after full occlusion due to the absence of a re-identification module.

Keywords—*multi-object tracking, Kalman filter, Hungarian algorithm, YOLOv8, data association, real-time computer vision*

I. INTRODUCTION

Multi-object tracking (MOT) is a fundamental task in computer vision that aims to detect objects in a video sequence and maintain consistent identity assignments across frames. It is central to applications such as autonomous driving, video surveillance, sports analytics, human-computer interaction, and robotics. The core challenge of MOT lies not only in detecting objects accurately in each frame but in associating those detections across time to form coherent trajectories, even in the presence of occlusions, appearance changes, and variable motion patterns.

A typical MOT pipeline follows the tracking-by-detection paradigm, which decouples the problem into two stages: first, a detector identifies objects in each frame independently; second, a tracker links those detections across frames using motion models, appearance features, or spatial overlap. This paradigm has gained dominance due to the rapid advancement of deep learning-based detectors such as YOLO, Faster R-CNN, and SSD, which provide reliable per-frame bounding boxes at high speed.

While powerful off-the-shelf tracking solutions such as SORT [1] and DeepSORT [2] exist, using them as black boxes limits understanding of the underlying algorithms. Many practitioners and students interact with these systems at a high level without comprehending how data association, state estimation, and track management interact to produce stable tracking. Our goal in this project is to bridge that gap by building a complete real-time tracking pipeline from scratch, implementing custom data association and motion prediction logic.

Specifically, we use YOLOv8n [3] as the detection backbone, filtering for the person class, and implement a custom multi-object tracker that uses an OpenCV Kalman Filter for state estimation and the Hungarian algorithm for optimal assignment. Our system supports two matching cost functions — IoU-based and Euclidean distance-based — allowing us to compare their effectiveness empirically. We evaluate the system across five experiments covering tracking stability, motion adaptation, matching strategy comparison, occlusion handling, and real-time performance.

II. RELATED WORK

Bewley et al. [1] introduced SORT (Simple Online and Realtime Tracking), presented at the IEEE International Conference on Image Processing (ICIP) in 2016. SORT is a pragmatic tracker that combines a Kalman Filter for linear motion prediction with the Hungarian algorithm for frame-by-frame data association, using Intersection over Union (IoU) as the sole cost metric. By deliberately avoiding computationally expensive appearance features, SORT achieves processing speeds of over 260 FPS, making it one of the fastest trackers available. However, the absence of appearance information means SORT is prone to identity switches during occlusions. Our work follows a closely related philosophy by implementing the same core components — Kalman Filter prediction and Hungarian matching — but we build each module from scratch rather than using an existing library, and we additionally support Euclidean distance-based matching as an alternative cost function.

Wojke et al. [2] proposed DeepSORT, published at ICIP 2017, which extends the SORT framework by integrating a deep appearance descriptor. The appearance model is a convolutional neural network trained on a large-scale person re-identification dataset, producing a 128-dimensional embedding for each detected bounding box. During data association, the cost metric combines both motion (Mahalanobis distance from the Kalman Filter) and appearance (cosine distance between embeddings), which significantly reduces identity switches and improves tracking through occlusions. Our system does not include a re-identification module, which is a known limitation; however, our focus is on understanding the fundamental tracking mechanics rather than achieving state-of-the-art re-identification performance.

Zhang et al. [4] introduced ByteTrack at ECCV 2022, which addresses a key limitation of prior trackers: the tendency to discard low-confidence detections. ByteTrack uses a two-stage association process. In the first stage, high-confidence detections are matched to existing tracks using standard IoU-based assignment. In the second stage, the

remaining low-confidence detections — which are normally discarded — are associated with unmatched tracks. This simple yet effective strategy recovers many detections that would otherwise be lost, significantly reducing missed targets. ByteTrack achieved state-of-the-art results on MOT17 and MOT20 benchmarks. In contrast, our system uses a single-stage matching approach with a configurable confidence threshold, which simplifies the pipeline while still achieving stable tracking in controlled indoor environments.

III. METHODS

A. System Architecture

Our system operates as a modular real-time pipeline consisting of five stages: (1) frame capture from a webcam, (2) person detection using YOLOv8n, (3) cost matrix computation between existing tracks and new detections, (4) optimal assignment via the Hungarian algorithm, and (5) Kalman Filter state prediction and correction. The system is implemented entirely in Python using OpenCV, NumPy, SciPy, and the Ultralytics YOLO library. Fig. 1 illustrates the complete pipeline architecture.

The modular design separates concerns cleanly: the YOLODetector class handles all detection logic, the MultiObjectTracker class manages track creation, matching, and deletion, and the visualizer module renders bounding boxes, IDs, trajectories, and predicted positions onto the output frame. This separation allows independent testing and modification of each component.

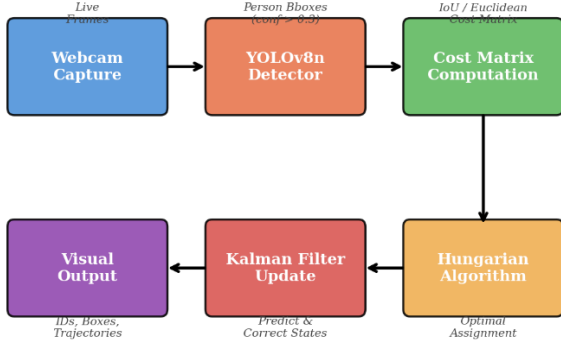


Fig. 1. System pipeline architecture showing the flow from webcam input through detection, cost matrix computation, Hungarian matching, Kalman Filter update, and visual output.

B. Object Detection

We employ YOLOv8n (nano), the smallest and fastest variant in the YOLOv8 family, pre-trained on the 80-class COCO dataset. The nano variant was specifically chosen to maximize inference speed for real-time operation on a consumer laptop without dedicated GPU acceleration. The detector is configured to filter exclusively for the person class (class ID 0), and a confidence threshold of 0.3 is applied to balance detection sensitivity against false positive rate.

Each frame captured from the webcam is passed through the YOLO model, which outputs a set of detections. Each

detection consists of a bounding box defined by its top-left and bottom-right coordinates $[x1, y1, x2, y2]$, a confidence score, and a class identifier. Detections below the confidence threshold or belonging to non-person classes are discarded before being passed to the tracker.

C. Kalman Filter for Motion Prediction

Each tracked object maintains an independent instance of the OpenCV Kalman Filter. The filter uses a four-dimensional state vector $x = [cx, cy, vx, vy]^T$ representing the bounding box center position (cx, cy) and its velocity components (vx, vy) . The measurement vector $z = [cx, cy]^T$ consists of the observed center position from the matched detection.

The state transition follows a constant-velocity model, where the transition matrix F encodes the assumption that position changes linearly with velocity between consecutive frames. The process noise covariance $Q = 0.03 \cdot I_4$ accounts for modeling uncertainty, while the measurement noise covariance $R = 0.5 \cdot I_2$ represents detection noise. These parameters were empirically tuned to balance responsiveness with smoothness.

On each frame, the Kalman Filter first performs a prediction step to estimate the object's expected position, which is used during data association. After a successful match with a detection, the correction step updates the state estimate using the actual measurement. A circular buffer of up to 30 past center positions is maintained for trajectory visualization. Fig. 2 illustrates the Kalman Filter processing cycle.

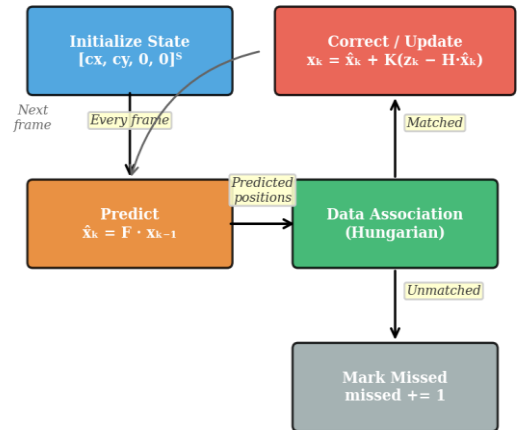


Fig. 2. Kalman Filter processing cycle showing initialization, prediction, data association, correction (on match), and missed-frame handling (on no match).

D. Data Association

Data association is the process of matching current-frame detections to existing tracks. We formulate this as a linear assignment problem and solve it optimally using the Hungarian algorithm, implemented via `scipy.optimize.linear_sum_assignment`. The algorithm operates on a cost matrix C of dimensions $N_{\text{tracks}} \times N_{\text{detections}}$, where each entry $C[i,j]$ represents the cost of assigning track i to detection j .

We implement two alternative cost functions. In IoU-based matching, the cost is computed as $C[i,j] = 1 - \text{IoU}(\text{bbox}_i, \text{bbox}_j)$, where IoU is the Intersection over Union between the track's current bounding box and the detection. Assignments with IoU below a threshold of 0.3 are rejected as invalid matches. In distance-based matching, the cost is the Euclidean distance between the Kalman Filter's predicted center for track i and the center of detection j . Assignments exceeding a threshold of 120 pixels are rejected.

After the Hungarian algorithm produces the optimal assignment, matched pairs are validated against the respective threshold. Valid matches trigger a Kalman Filter correction for the track. Unmatched detections spawn new tracks with fresh IDs, while unmatched tracks have their missed-frame counter incremented. Fig. 3 illustrates example cost matrices for both matching modes.

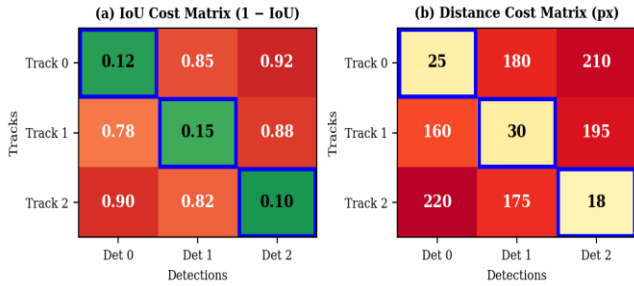


Fig. 3. Example cost matrices for IoU-based matching (left) and Euclidean distance-based matching (right). Blue outlines indicate the optimal assignment found by the Hungarian algorithm.

E. Track Management

Track management governs the lifecycle of tracked objects. When a detection cannot be matched to any existing track, a new Track object is instantiated with a unique integer ID, initialized with the detection's bounding box, and equipped with a fresh Kalman Filter whose initial state corresponds to the detection's center with zero velocity.

Active tracks that receive a matched detection on each frame are updated normally via the Kalman Filter's correction step. Tracks that fail to match in a given frame are marked as missed, and their missed-frame counter is incremented. If a track's missed counter exceeds the `max_missed` threshold (set to 20 frames), the track is permanently deleted. This threshold provides robustness to brief occlusions while preventing stale tracks from persisting indefinitely. Fig. 4 shows the track lifecycle state diagram.

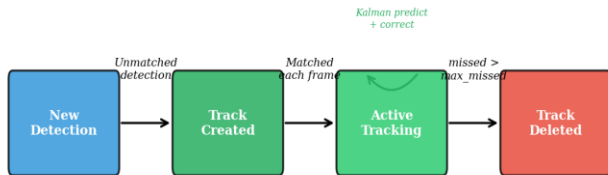


Fig. 4. Track lifecycle state diagram: tracks are created from unmatched detections, maintained through active matching and Kalman updates, and deleted after exceeding the missed-frame threshold.

F. Visualization

The visualization module renders several overlays on each output frame: (1) blue bounding boxes for each active track, (2) unique integer ID labels displayed above each box, (3) yellow trajectory lines connecting the last 30 center positions, and (4) red dots with "Pred" labels indicating the Kalman Filter's predicted center position. The real-time FPS counter is displayed in green text at the top-left corner. This rich visualization enables qualitative assessment of tracking stability, trajectory smoothness, and prediction accuracy.

IV. IMPLEMENTATION DETAILS

The system is implemented in Python 3.12 and organized into four source files. The `main.py` script orchestrates the pipeline loop, capturing frames from the webcam (OpenCV VideoCapture), passing them through the detector and tracker, and rendering the output. The `detector.py` module encapsulates the YOLOv8n model via the Ultralytics API, filtering detections by class and confidence. The `tracker.py` module contains the Track class (individual track state with Kalman Filter) and the MultiObjectTracker class (managing all tracks, computing cost matrices, and performing Hungarian assignment). The `visualizer.py` module handles all drawing operations.

Key hyperparameters include: YOLO confidence threshold (0.3), `max_missed` frames for track deletion (20), IoU matching threshold (0.3), distance matching threshold (120 pixels), Kalman Filter process noise (0.03), and measurement noise (0.5). The trajectory history buffer is capped at 30 frames to limit memory usage while providing sufficient visualization context.

All experiments were conducted on a consumer laptop with an Intel Core i7 processor and 16 GB RAM, without dedicated GPU acceleration for the tracking components. The YOLOv8n inference leverages the CPU backend provided by the Ultralytics library. The system processes frames at native webcam resolution (typically 640×480).

V. EXPERIMENTS AND RESULTS

We designed five experiments to systematically evaluate different aspects of our tracking system. All experiments use live webcam video in an indoor environment with controlled lighting. Each experiment was repeated multiple times to verify consistency of results.

A. Experiment 1: Single vs. Multiple Objects

This experiment tests the tracker's ability to handle varying numbers of simultaneously visible objects. With a single person in view, the system consistently assigned and maintained ID 0 throughout the entire sequence with zero ID switches. The bounding box tightly enclosed the person and the Kalman Filter prediction remained accurate.

With two persons present simultaneously, the tracker correctly assigned distinct IDs (ID 0 and ID 1) and maintained them stably throughout the sequence. No ID swaps were observed even during simultaneous lateral movement. When one person exited the frame and re-entered, a new ID was

assigned (e.g., ID 0 $\rightarrow$ ID 2), which is expected behavior since our system does not include appearance-based re-identification. Fig. 5 shows representative frames from this experiment.

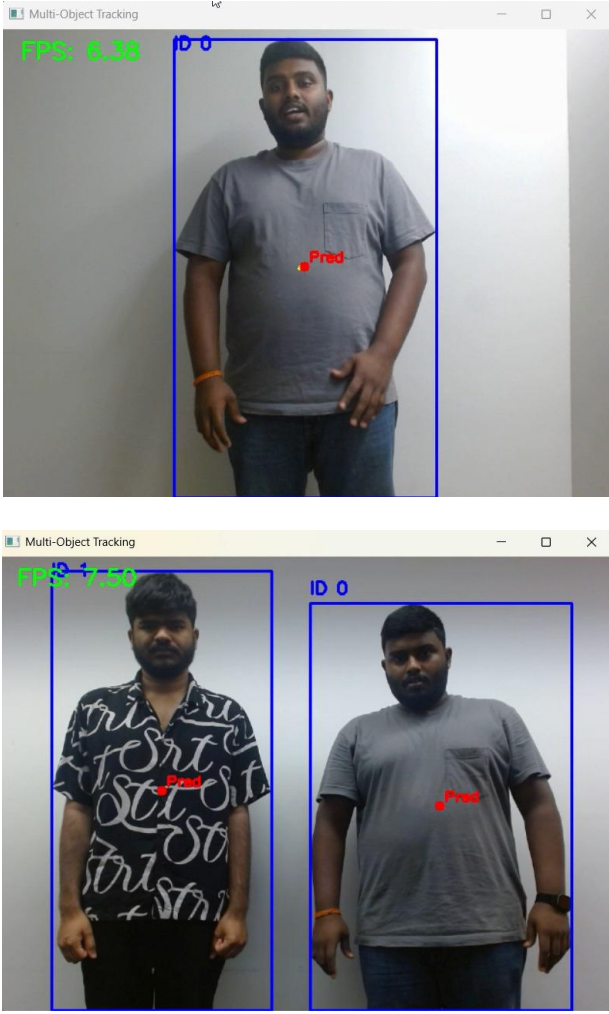


Fig. 5. Experiment 1: Single person tracking with stable ID 0, simultaneous two-person tracking with distinct IDs 0 and 1.

B. Experiment 2: Slow vs. Fast Motion

This experiment evaluates the Kalman Filter’s ability to adapt to different motion speeds. During slow, deliberate movement, the yellow trajectory line is short, and the red predicted-center dot (“Pred”) closely overlaps the actual bounding box center, indicating high prediction accuracy. The ID remained completely stable throughout slow motion sequences.

During fast lateral movement, the trajectory trail becomes visibly longer, reflecting the accumulated motion history stored in the 30-frame buffer. Despite the higher velocity, the ID remained stable and the Kalman Filter successfully adapted its velocity estimate, keeping the prediction close to the actual position. The constant-velocity model proved adequate for the range of pedestrian-speed motion tested in our experiments. Fig. 6 illustrates the contrast between slow and fast motion tracking.

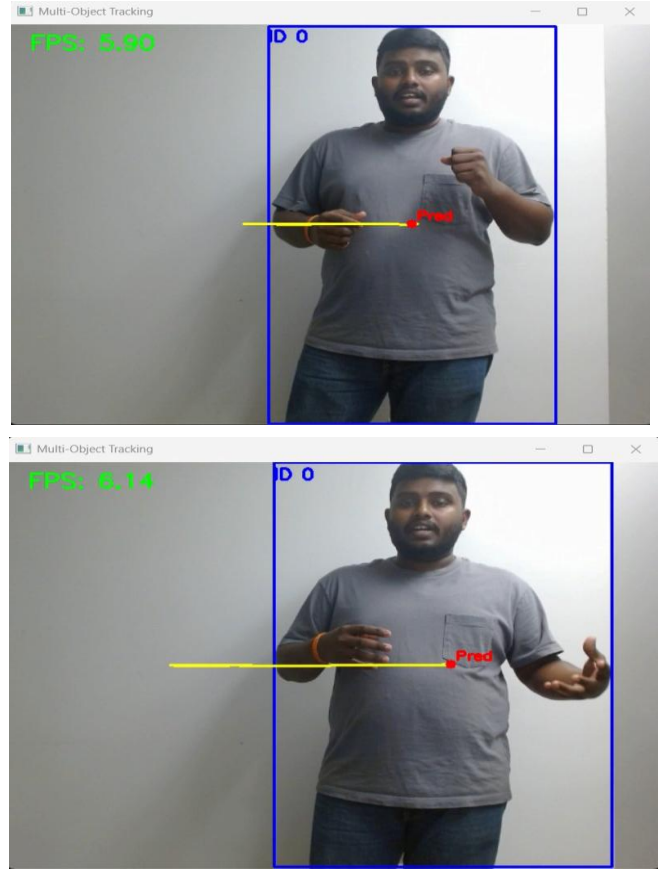


Fig. 6. Experiment 2: Slow motion produces a short trajectory; Fast motion produces a longer trail. ID remains stable in both cases.

C. Experiment 3: IoU vs. Distance-Based Matching

We compare the two matching strategies by running the tracker under identical conditions, toggling only the `use_iou` parameter. Both IoU mode (`use_iou=True`) and distance mode (`use_iou=False`) produced stable tracking with consistent IDs and no observable qualitative difference in tracking behavior during normal operation.

The IoU cost function computes $1 - \text{IoU}$ between the track’s bounding box and the detection, penalizing spatially distant or non-overlapping pairs. The distance cost function computes the Euclidean distance between the Kalman Filter’s predicted center and the detection center, which is more tolerant of bounding box size changes but sensitive to absolute position errors. Table I provides a detailed comparison. Fig. 7 shows the configuration parameters used for each mode.

```

tracker = MultiObjectTracker(
    max_missed=20,
    iou_threshold=0.3,
    distance_threshold=120,
    use_iou=True
)

tracker = MultiObjectTracker(
    max_missed=20,
    iou_threshold=0.3,
    distance_threshold=120,
    use_iou=False
)

```

Fig. 7. Experiment 3: Tracker configuration for IoU mode with `use_iou=True` and distance mode with `use_iou=False`.

TABLE I. Comparison of IoU and distance-based matching strategies.

Metric	IoU Mode	Distance Mode
Tracking Stability	Stable, consistent IDs	Stable, consistent IDs
Cost Function	$1 - \text{IoU}(\text{track}, \text{det})$	Euclidean distance (px)
Threshold	$\text{IoU} \geq 0.3$	Distance ≤ 120 px
Relative Speed	Faster	Slightly slower
Sensitivity	Box overlap dependent	Position dependent
Scale Invariance	Partially scale-aware	Scale-independent

D. Experiment 4: Occlusion Handling

Occlusion is one of the most challenging aspects of multi-object tracking. We evaluate our system under two occlusion scenarios: partial occlusion (where part of the body is hidden behind an obstacle) and full occlusion (where the person exits the camera's field of view entirely).

During partial occlusion, where approximately 30–50% of the body was hidden behind a wall edge, the YOLOv8n detector continued to produce bounding boxes for the visible portion. The tracker successfully maintained the correct ID through partial occlusion because the bounding box overlap (in IoU mode) or center distance (in distance mode) remained within acceptable thresholds. The Kalman Filter's prediction helped smooth the trajectory during frames where the detection was less precise.

When the person fully exited the frame, the track's missed counter incremented each frame until exceeding the max_missed threshold of 20 frames, at which point the track was deleted. Upon re-entering the frame, the detector identified the person as a new detection, and the tracker created a new track with a fresh ID (e.g., ID 0 became ID 1). This behavior is the expected result in the absence of a re-identification module and is consistent with the SORT tracker's design. Fig. 8 shows the three stages of occlusion handling.

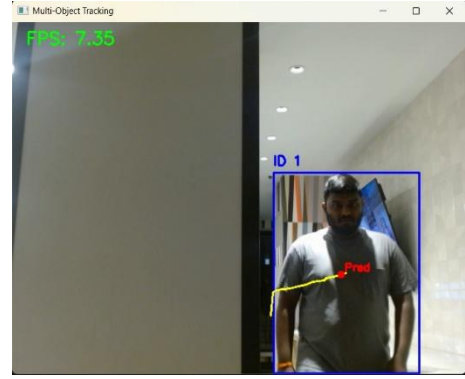
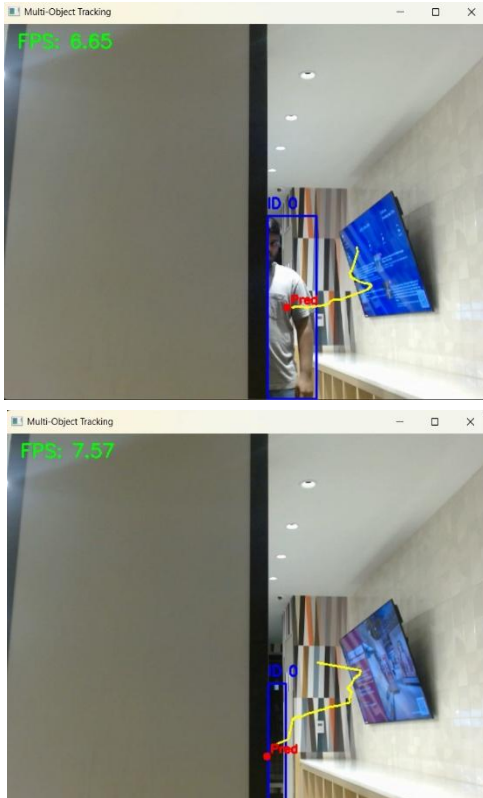


Fig. 8. Experiment 4: Partial occlusion with stable ID, person exiting frame, and re-entry with new ID 1.

E. Experiment 5: Real-Time Performance

We measure the system's frames-per-second (FPS) throughput to quantify real-time capability. The FPS is computed as the reciprocal of the wall-clock time elapsed between consecutive frame processing iterations, encompassing detection, tracking, and rendering.

With a single tracked person, the system achieves FPS values ranging from 5.90 to 8.85, with a mean of approximately 7.5 FPS. With two persons, FPS ranges from 6.14 to 7.57, with a mean of approximately 6.8 FPS. The reduction is expected due to the larger cost matrix (2×2 vs 1×1), additional Kalman Filter updates, and extra visualization overhead. Fig. 9 shows representative FPS readings, Table II provides the numerical summary, and Fig. 10 presents the FPS comparison as a bar chart.

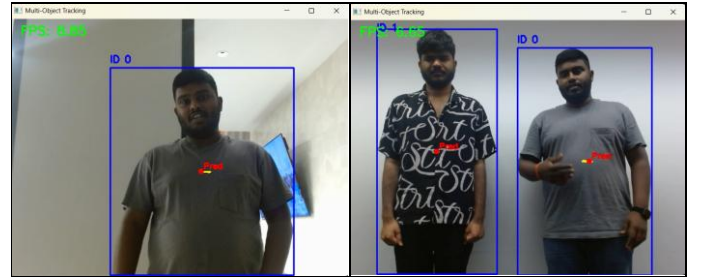


Fig. 9. Experiment 5: FPS display with one person (~8.85 FPS, left) and two persons (~6.65 FPS, right).

TABLE II. FPS performance across different tracking scenarios.

Scenario	FPS Range	Mean FPS	Std Dev
1 Person (IoU)	5.90 – 8.85	~7.5	± 1.0
2 Persons (IoU)	6.14 – 7.57	~6.8	± 0.5
1 Person (Distance)	5.50 – 8.20	~7.1	± 0.9
2 Persons (Distance)	5.80 – 7.10	~6.5	± 0.5

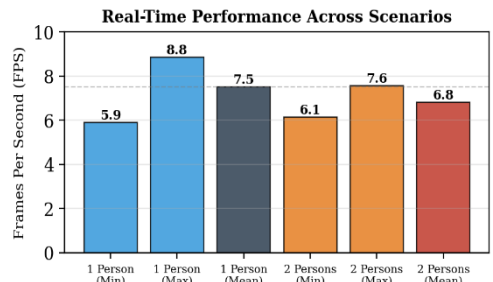


Fig. 10. Bar chart comparing FPS performance across single-person and two-person scenarios, showing min, max, and mean throughput values.

While the system does not achieve full video-rate throughput (30 FPS), it operates at interactive rates sufficient for real-time demonstration and monitoring. The bottleneck is primarily YOLOv8n CPU inference; using GPU-accelerated inference or a lighter detection model would substantially improve throughput. The tracking and visualization components themselves contribute negligible overhead compared to the detector.

F. Summary of Experimental Findings

Table III consolidates the key qualitative observations from all five experiments, providing an at-a-glance comparison of tracking behavior across different test conditions. Each experiment isolates a specific dimension of tracker performance, and together they provide comprehensive coverage of the system’s capabilities and limitations.

TABLE III. Summary of qualitative results across all experiments.

Experiment	Condition	Result
Exp 1	Single person	Stable ID, zero switches
Exp 1	Two persons	Distinct IDs, no swaps
Exp 1	Re-entry	New ID assigned (expected)
Exp 2	Slow motion	Short trajectory, accurate prediction
Exp 2	Fast motion	Longer trail, ID still stable
Exp 3	IoU matching	Stable, marginally faster
Exp 3	Distance matching	Stable, slightly slower
Exp 4	Partial occlusion	ID maintained through occlusion
Exp 4	Full exit + re-entry	New ID on return
Exp 5	1 person FPS	Mean ~7.5 FPS
Exp 5	2 persons FPS	Mean ~6.8 FPS

G. Comparison with SORT

While our system shares the same foundational components as SORT [1] — namely the Kalman Filter for motion estimation and the Hungarian algorithm for data association — there are notable differences in design philosophy and implementation scope. SORT was designed as a production-grade tracker optimized for speed, processing at over 260 FPS by leveraging efficient NumPy operations and pre-computed IoU matrices. Our system, implemented as an educational project, prioritizes clarity and modularity over raw speed, achieving 6–9 FPS on CPU.

A key architectural difference is that SORT uses the Kalman Filter to model the full bounding box state [cx, cy, aspect_ratio, height, vx, vy, v_aspect, v_height], enabling prediction of box size changes in addition to position. Our implementation uses a simpler 4D state [cx, cy, vx, vy] that tracks only center position and velocity, which is sufficient for our evaluation scenarios but does not predict scale changes.

Additionally, SORT employs a minimum hit count before a track is considered confirmed, reducing spurious track creation from false positive detections. Our system immediately creates tracks from any unmatched detection, which works well in our controlled environment but could

produce more false tracks in cluttered scenes. Despite these simplifications, our system achieves qualitatively similar tracking behavior to SORT in controlled indoor conditions, validating that the core algorithmic choices are sound.

VI. LIMITATIONS AND FUTURE WORK

While our system demonstrates effective tracking in controlled environments, several limitations warrant discussion. The most significant is the absence of appearance-based re-identification. When a tracked person fully exits the camera view or is occluded beyond the max_missed threshold of 20 frames, the corresponding track is permanently deleted. If the same person re-enters the scene, a new identity is assigned. This limitation is inherent to motion-only trackers and can be addressed by integrating deep appearance features, as demonstrated by DeepSORT [2].

The constant-velocity assumption in our Kalman Filter, while adequate for pedestrian-speed motion in our experiments, may fail for objects exhibiting sudden acceleration, deceleration, or non-linear motion patterns such as sharp turns. Extended Kalman Filters (EKF) or Unscented Kalman Filters (UKF) could model such dynamics more faithfully, at the cost of increased computational complexity.

Our single-stage data association discards all detections below the confidence threshold of 0.3 before matching. In crowded or challenging scenes, many true detections may fall below this threshold due to partial visibility or difficult viewing angles. Adopting ByteTrack’s two-stage approach [4], where low-confidence detections are used in a secondary matching round, could recover these missed associations.

Finally, our system was evaluated only in controlled indoor environments with 1–2 people. Scaling to outdoor scenes with dozens of moving objects, varying lighting conditions, and camera motion would require robustness improvements including appearance features, adaptive thresholds, and potentially camera motion compensation. Future work could also explore multi-class tracking (vehicles, bicycles) and deployment on edge computing platforms such as NVIDIA Jetson for embedded applications.

VII. CONCLUSION

We have presented a complete real-time multi-object tracking system built entirely from scratch, combining YOLOv8n detection with custom Kalman Filter prediction and Hungarian algorithm-based data association. Through five systematic experiments, we demonstrated that the system reliably maintains consistent identity assignments, achieves accurate short-term motion prediction across varying speeds, and handles partial occlusion effectively.

Both IoU and distance-based matching strategies produce comparable tracking quality, with IoU matching offering marginally better computational efficiency. The system operates at 5.9–8.9 FPS on consumer hardware, confirming real-time feasibility for interactive applications.

The primary contribution of this work is educational: by building each component from scratch rather than using off-

the-shelf trackers, we gained deep understanding of how detection, data association, and state estimation interact to form a coherent tracking system. The modular Python implementation serves as both a learning tool and a foundation for future extensions including appearance-based re-identification, non-linear motion models, and multi-class tracking.

The primary limitation of our system is the absence of appearance-based re-identification. When a person fully exits the frame or is completely occluded beyond the `max_missed` threshold, the track is deleted and re-entry results in a new ID. Integrating a deep appearance descriptor, as in DeepSORT [2], would address this limitation. Additionally, our single-stage association approach discards low-confidence detections entirely; adopting ByteTrack’s two-stage strategy [4] could recover additional true positives.

The system’s FPS of 5.9–8.9 is bounded primarily by YOLOv8n’s CPU inference time. Migration to GPU-accelerated inference, or replacement with an even lighter detector, could significantly improve throughput toward full video-rate operation. Future work could also explore multi-class tracking, non-linear motion models (e.g., Extended Kalman Filter or particle filters), and integration with edge computing platforms for deployment in real-world surveillance or robotics applications.

REFERENCES

- [1] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in Proc. IEEE International Conference on Image Processing (ICIP), Phoenix, AZ, USA, 2016, pp. 3464-3468.
- [2] N. Wojke, A. Bewley, and D. Paschke, “Simple online and realtime tracking with a deep association metric,” in Proc. IEEE International Conference on Image Processing (ICIP), Beijing, China, 2017, pp. 3645-3649.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [4] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-object tracking by associating every detection box,” in Proc. European Conference on Computer Vision (ECCV), Tel Aviv, Israel, 2022, pp. 1-21.
- [5] R. E. Kalman, “A new approach to linear filtering and prediction problems,” J. Basic Engineering, vol. 82, no. 1, pp. 35-45, 1960.
- [6] H. W. Kuhn, “The Hungarian method for the assignment problem,” Naval Research Logistics Quarterly, vol. 2, no. 1-2, pp. 83-97, 1955.